МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Институт математики и информационных систем
Центр образовательных программ топ-уровня в сфере искусственного интеллекта

Методические указания к лабораторной работе по дисциплине:

«Основы машинного обучения»

Лабораторная работа № 1

«Полный цикл решения задачи классификации. Линейные модели и стохастический градиент»

Киров — 2026

СОДЕРЖАНИЕ

1. Общая характеристика лабораторной работы
2. Цель и планируемые результаты
3. Необходимые теоретические сведения
4. Программное обеспечение и воспроизводимость
5. Исходные данные и варианты
6. Подготовка к выполнению работы
7. Пример выполнения работы
8. Задание по индивидуальному варианту
9. Контрольные вопросы
10. Требования к отчёту
11. Критерии оценивания
12. Существенные ошибки и основания для доработки
13. Дополнительное задание

Приложение А. Таблица вариантов

Приложение Б. Минимальный шаблон отчёта

1. Общая характеристика лабораторной работы

Тема

Полный цикл решения задачи классификации. Линейные модели и стохастический градиентный спуск (SGD).

Работа соответствует вводной лекции и лекции № 2 курса по основам машинного обучения. В ней объединены схема «объекты → признаки → функция потерь → оптимизация», методология корректного эксперимента, минимизация эмпирического риска, отступ, логистическая функция потерь, линейные классификаторы, метрики качества и L2-регуляризация.

Тип задачи

Обучение с учителем, классификация. Библиотечная часть выполняется для исходной задачи варианта — бинарной или многоклассовой. Для собственной реализации SGD многоклассовая задача сводится к бинарной схеме «один класс против остальных».

Сквозная схема работы

1. Получить и описать набор данных своего варианта;
2. Зафиксировать экспериментальный протокол и отделить тестовую выборку;
3. Построить простую базовую модель;
4. Реализовать обучение логистической регрессии методом SGD;
5. Собрать безопасный preprocessing pipeline;
6. Подобрать гиперпараметры на кросс-валидации;
7. Однократно оценить зафиксированные модели на тестовой выборке;
8. Проанализировать ошибки и сформулировать выводы.

Основной принцип. Цель работы состоит не в получении максимальной метрики любой ценой, а в построении корректного, воспроизводимого и объяснимого ML-эксперимента.

2. Цель и планируемые результаты

Цель работы

Изучить полный цикл решения задачи классификации и приобрести практические навыки подготовки данных, реализации базового алгоритма SGD, построения линейных моделей, подбора гиперпараметров и корректного оценивания качества.

Планируемые результаты

После выполнения работы студент должен уметь:

- формулировать задачу классификации в терминах объектов, признаков, классов, модели и функции потерь;
- различать функцию потерь и итоговую метрику качества;
- исследовать типы признаков, пропуски и распределение классов;

- разделять данные без утечки информации и сохранять тестовую выборку для финальной оценки;
- строить ColumnTransformer и Pipeline для числовых и категориальных признаков;
- реализовывать логистическую потерю, её градиент и шаг SGD;
- объяснять влияние темпа обучения и регуляризации;
- использовать DummyClassifier, SGDClassifier и LogisticRegression;
- применять StratifiedKFold и GridSearchCV только к обучающей части;
- вычислять Accuracy, Precision, Recall, F1 и ROC-AUC;
- интерпретировать confusion matrix и анализировать ошибки модели;
- формулировать выводы на основании полученных численных результатов.

3. Необходимые теоретические сведения

Постановка задачи классификации

Пусть X — пространство объектов, Y — конечное множество классов, а обучающая выборка состоит из пар (x_i, y_i), $i = 1, \ldots, \ell$. Требуется построить алгоритм $a(x)$, который по признакам нового объекта прогнозирует его класс.

$$g(x) = w^T x + b, \quad a(x) = \text{sign}(g(x))$$

В бинарном случае вектор w задаёт ориентацию разделяющей гиперплоскости, а b — её смещение. В многоклассовой задаче строятся несколько линейных оценок, после чего выбирается класс с наибольшим значением оценки.

Минимизация эмпирического риска

Обучение параметров модели формулируется как минимизация среднего значения функции потерь на обучающей выборке:

$$Q(w, b) = (1/\ell) \cdot \Sigma_i \, L(a(x_i), y_i) \to \min.$$

Функция потерь L оценивает ошибку на одном объекте. Метрика качества, напротив, используется для содержательного сравнения моделей и не обязана быть удобной для дифференцирования. Например, логистическая потеря пригодна для оптимизации, а F1 — для оценивания качества классификации.

Отступ и логистическая функция потерь

Для бинарной классификации метки кодируются как $y_i \in \{-1, +1\}$. Отступ объекта равен:

$$M_i = y_i \cdot (w^T x_i + b).$$

Положительный отступ соответствует правильной классификации. Чем больше его значение, тем дальше объект находится от границы решения. Для логистической регрессии применяется гладкая убывающая функция потерь:

$$L(M) = \ln(1 + \exp(-M)).$$

Логистическая регрессия также имеет вероятностную интерпретацию: сигмоида $\sigma(z) = 1/(1 + \exp(-z))$ переводит линейную оценку в вероятность положительного класса.

Градиентный спуск и SGD

Полный градиентный спуск вычисляет градиент по всей выборке на каждом шаге. SGD обновляет параметры по одному объекту или мини-пакету. Один шаг становится дешевле, но траектория оптимизации приобретает шумовой характер.

$$w \leftarrow w - \eta \cdot \nabla Q(w),$$

где η — темп обучения (learning rate). Слишком малое η приводит к медленной сходимости, а слишком большое — к колебаниям или расходимости.

L2-регуляризация

Регуляризация ограничивает сложность модели. В данной работе используется критерий:

$$Q\tau(w, b) = (1/\ell) \cdot \Sigma_i L(M_i) + (\tau/2) \cdot ||w||^2 \rightarrow \min.$$

Для одного объекта градиент по весам имеет вид:

$$\nabla w\, L = -y_i x_i/(1 + \exp(M_i)) + \tau w.$$

Свободный коэффициент b обычно не регуляризуется. Увеличение τ сокращает норму весов, но чрезмерная регуляризация способна привести к недообучению.

Экспериментальный протокол

Тестовая выборка должна оставаться закрытой до завершения выбора признаков, способа preprocessing, семейства модели и гиперпараметров. Подбор выполняется только на обучающей части посредством стратифицированной кросс-валидации.

> **Утечка данных (data leakage).** Импутация, масштабирование или кодирование, выполненные на всей выборке до разбиения, передают модели информацию о тестовых объектах и делают оценку качества завышенной. Преобразования следует включать в Pipeline.

Метрики классификации

Таблица 1 — Основные метрики классификации

Метрика	Определение	Интерпретация
Accuracy	Доля верных ответов	Информативна при сопоставимых размерах классов.
Precision	TP/(TP + FP)	Доля истинных положительных среди положительных прогнозов.
Recall	TP/(TP + FN)	Доля найденных положительных объектов.
F1	2PR/(P + R)	Гармоническое среднее Precision и Recall.
ROC-AUC	Площадь под ROC-кривой	Качество ранжирования объектов при всех порогах.

Для сопоставимости вариантов основной метрикой подбора является macro-F1: F1 вычисляется отдельно для каждого класса, затем значения усредняются с равным весом.

4. Программное обеспечение и воспроизводимость

Программное обеспечение

- Python 3.11 или совместимая версия;
- Jupyter Notebook / JupyterLab либо VS Code с расширением Jupyter;
- NumPy, Pandas, Matplotlib;
- scikit-learn;
- доступ к OpenML или локальная копия набора данных.

Листинг 1 — Создание окружения и установка библиотек

```
# Создаём отдельное окружение, чтобы версии библиотек не конфликтовали
# с другими Python-проектами на компьютере.
python -m venv .venv

# Активируйте окружение одной из следующих команд.
# Windows:
.venv\Scripts\activate

# Linux / macOS:
source .venv/bin/activate

# Устанавливаем только библиотеки, необходимые для этой работы.
pip install numpy pandas matplotlib scikit-learn openml jupyter
```

Фиксация версий и случайности

Листинг 2 — Версии библиотек

```
# Версии следует сохранить в начале notebook: это упрощает
# воспроизведение результатов на другом компьютере.
import sys
import matplotlib
import numpy as np
import pandas as pd
import sklearn

# Печатаем версии среды и основных библиотек одним блоком.
print("Python:", sys.version)
print("NumPy:", np.__version__)
print("Pandas:", pd.__version__)
print("Matplotlib:", matplotlib.__version__)
print("scikit-learn:", sklearn.__version__)
```

Во всех операциях, содержащих случайность, используется единое значение:

$$\text{RANDOM_STATE} = 1000 + \text{номер_варианта}.$$

Перед сдачей notebook необходимо выполнить Restart Kernel → Run All. Код должен работать последовательно, без скрытых переменных, абсолютных путей и ручного восстановления промежуточных результатов.

5. Исходные данные и варианты

Общий принцип

Каждому студенту назначается вариант 1–30. Вариант определяет набор данных OpenML. К методическим указаниям прилагается архив ML_Lab1_Datasets.zip. После распаковки

каталог ML_Lab1_Datasets должен находиться рядом с notebook. Имя CSV-файла указано в четвёртом столбце таблицы вариантов, а имя целевого столбца — в manifest.csv. В отчёте обязательно указываются номер варианта, data_id, название набора и фактически использованный источник данных.

Загрузка OpenML-варианта

Листинг 3 — Общая заготовка загрузки варианта

```
from sklearn.datasets import fetch_openml

# Укажите номер своего варианта и соответствующий data_id из приложения А.
VARIANT = ...
DATA_ID = ...
RANDOM_STATE = 1000 + VARIANT

# as_frame=True возвращает признаки и целевую переменную как объекты Pandas.
data = fetch_openml(data_id=DATA_ID, as_frame=True)
X = data.data.copy()
y = data.target.copy()

# Контрольные сообщения помогают сразу заметить ошибочный data_id.
print("Название:", data.details.get("name"))
print("data_id:", DATA_ID)
print("Размер X:", X.shape)
print("Число классов:", y.nunique())
```

Загрузка локальной копии

Листинг 4 — Загрузка CSV-файла по манифесту

```
from pathlib import Path
import pandas as pd

# Архив необходимо заранее распаковать рядом с notebook.
DATA_DIR = Path("ML_Lab1_Datasets")
VARIANT = ...
RANDOM_STATE = 1000 + VARIANT

# Манифест связывает номер варианта, имя файла и целевой столбец.
manifest = pd.read_csv(DATA_DIR / "manifest.csv")
row = manifest.loc[manifest["variant"] == VARIANT].iloc[0]

data_path = DATA_DIR / row["file_name"]
target_column = row["target_column"]

# В каждом CSV целевой столбец уже включён в общую таблицу.
frame = pd.read_csv(data_path)
X = frame.drop(columns=[target_column])
y = frame[target_column]

print("Файл:", data_path)
print("Размер X:", X.shape)
print("Целевой столбец:", target_column)
```

Карточка набора данных

Таблица 2 — Карточка набора данных

Параметр	Значение
Номер варианта	

Параметр	Значение
Название набора	
OpenML data_id	
Число объектов	
Число исходных признаков	
Числовые признаки	
Категориальные признаки	
Число классов	
Распределение классов	
Число пропусков	
Особенности набора	

6. Подготовка к выполнению работы

До начала основной части необходимо:

1. повторить понятия ERM, градиентного спуска, SGD, отступа, логистической потери и регуляризации;
2. понимать различие между параметрами модели и гиперпараметрами;
3. понимать назначение Train, Validation/CV и Test;
4. подготовить работоспособное программное окружение;
5. знать номер варианта и способ загрузки набора данных.

Входной самоконтроль

До выполнения эксперимента студент должен уметь кратко объяснить: что минимизирует логистическая регрессия; чем SGD отличается от полного градиента; почему нельзя подбирать гиперпараметры по test; зачем масштабировать числовые признаки.

7. Пример выполнения работы

Ниже приведён сквозной пример на наборе Titanic (OpenML data_id 40945). Этот набор не входит в таблицу индивидуальных вариантов и содержит числовые, категориальные признаки и пропуски. Целевой признак survived принимает значения 0 и 1.

> **Назначение примера.** Код следует выполнить последовательно, разобраться в назначении каждой ячейки, а затем адаптировать к набору своего варианта. Численные результаты и выводы для индивидуального варианта должны быть получены самостоятельно.

Импорт библиотек и параметры эксперимента

Листинг 5 — Импорт библиотек

```
# Скрываем только предупреждения о будущих изменениях API;
# остальные предупреждения оставляем видимыми.
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)

# Базовые библиотеки для вычислений, таблиц и графиков.
from pathlib import Path
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Компоненты scikit-learn сгруппированы по назначению.
from sklearn.base import clone
from sklearn.compose import ColumnTransformer
from sklearn.datasets import fetch_openml
from sklearn.dummy import DummyClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression, SGDClassifier
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    accuracy_score,
    classification_report,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
)
from sklearn.model_selection import (
    GridSearchCV,
    StratifiedKFold,
    cross_validate,
    train_test_split,
)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# Все случайные операции используют одно значение для воспроизводимости.
RANDOM_STATE = 42            # только для учебного примера
TEST_SIZE = 0.20
SCORING = "f1_macro"
```

Для своего варианта. Замените RANDOM_STATE на 1000 + номер варианта. Остальные константы сохраняются, если в задании не указано иное.

Загрузка и первичный анализ данных

Листинг 6 — Загрузка учебного набора Titanic

```
# Сначала пытаемся использовать приложенную локальную копию.
example_path = Path("ML_Lab1_Datasets") / "example_titanic_40945.csv"

if example_path.exists():
    frame = pd.read_csv(example_path)
    print("Источник: локальная копия", example_path)
else:
    # Резервный вариант нужен, если архив с данными не был распакован.
    data = fetch_openml(data_id=40945, as_frame=True)
    frame = data.data.copy()
    frame["survived"] = data.target
    print("Источник: OpenML, data_id 40945")

# Для учебного примера оставляем семь содержательных признаков разных типов.
features = [
    "pclass", "sex", "age", "sibsp",
```

```
    "parch", "fare", "embarked",
]
X = frame[features].copy()
y = pd.to_numeric(frame["survived"]).astype("int64")

# Проверяем размерность, классы и первые строки до дальнейшей обработки.
print("Размер X:", X.shape)
print("Классы:", sorted(y.unique()))
display(X.head())
```

Листинг 7 — Диагностика типов и пропусков

```
# Карточка признаков объединяет тип, число значений и статистику пропусков.
data_card = pd.DataFrame({
    "dtype": X.dtypes.astype(str),
    "n_unique": X.nunique(dropna=True),
    "n_missing": X.isna().sum(),
    "missing_share": X.isna().mean().round(3),
})

# Отдельно оцениваем абсолютный и относительный размер каждого класса.
class_table = pd.DataFrame({
    "count": y.value_counts().sort_index(),
    "share": y.value_counts(normalize=True).sort_index().round(3),
})

# display удобнее print для просмотра таблиц внутри Jupyter Notebook.
display(data_card)
display(class_table)
```

Листинг 8 — Две базовые визуализации

```
# На одном рисунке показываем баланс классов и распределение возраста.
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

# Столбчатая диаграмма позволяет быстро увидеть дисбаланс классов.
y.value_counts().sort_index().plot.bar(ax=axes[0], color="0.45")
axes[0].set_title("Распределение классов")
axes[0].set_xlabel("survived")
axes[0].set_ylabel("Число объектов")

plot_frame = pd.DataFrame({"age": X["age"], "survived": y})
# Boxplot показывает медиану, разброс и возможные выбросы по каждому классу.
plot_frame.boxplot(column="age", by="survived", ax=axes[1])
axes[1].set_title("Возраст по классам")
axes[1].set_xlabel("survived")
axes[1].set_ylabel("Возраст")

# Pandas добавляет общий заголовок автоматически — убираем его перед выводом.
plt.suptitle("")
plt.tight_layout()
plt.show()
```

В кратком комментарии к EDA следует указать число объектов и признаков, долю пропусков, соотношение классов и не менее двух потенциальных сложностей данных. Для Titanic такими сложностями являются пропуски и совместная обработка признаков разных типов.

Для своего варианта. Не копируйте список features. Используйте все содержательные признаки своего набора, предварительно проверив типы данных, константные столбцы и возможные идентификаторы объектов.

Разбиение данных и кросс-валидация

Листинг 9 — Фиксация тестовой выборки

```
# Тестовая выборка отделяется один раз и остаётся закрытой до финальной оценки.
# stratify сохраняет приблизительные доли классов в обеих частях.
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=TEST_SIZE,
    stratify=y,
    random_state=RANDOM_STATE,
)

# Кросс-валидация применяется только внутри обучающей части.
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=RANDOM_STATE,
)

# Размерности помогают проверить, что разбиение выполнено ожидаемым образом.
print("Train:", X_train.shape)
print("Test:", X_test.shape)
```

Правило test. После этой ячейки X_test и y_test не используются до подраздела «Финальная оценка на test». Все решения о preprocessing, признаках и гиперпараметрах принимаются только по X_train, y_train и результатам CV.

Базовая модель

Листинг 10 — Оценивание DummyClassifier на обучающей части

```
# most_frequent всегда предсказывает самый частый класс и не использует признаки.
baseline = DummyClassifier(strategy="most_frequent")

# Используем ту же CV-схему, что и для основных моделей.
baseline_cv = cross_validate(
    baseline,
    X_train,
    y_train,
    cv=cv,
    scoring={"accuracy": "accuracy", "f1_macro": "f1_macro"},
)

# Усредняем метрики по пяти фолдам для компактного сравнения.
baseline_summary = pd.Series({
    "accuracy_cv_mean": baseline_cv["test_accuracy"].mean(),
    "f1_macro_cv_mean": baseline_cv["test_f1_macro"].mean(),
})
display(baseline_summary.round(4))
```

Baseline показывает уровень, достижимый без использования признаков. Основная модель должна сравниваться с baseline при одной и той же схеме оценивания.

Подготовка данных для собственной реализации SGD

Для демонстрации механизма алгоритма используются четыре числовых признака. Внутреннее разбиение train_demo / val_demo формируется только из X_train.

Листинг 11 — Внутреннее разбиение, импутация и масштабирование

```
# Для прозрачной реализации SGD используем только четыре числовых признака.
manual_features = ["age", "fare", "sibsp", "parch"]

# Validation формируется только из X_train: тестовая часть остаётся закрытой.
X_demo_train, X_demo_val, y_demo_train, y_demo_val = train_test_split(
    X_train[manual_features],
    y_train,
    test_size=0.20,
    stratify=y_train,
    random_state=RANDOM_STATE,
)

# Медиана устойчива к выбросам, а StandardScaler выравнивает масштабы признаков.
imputer = SimpleImputer(strategy="median")
scaler = StandardScaler()

# fit выполняется только на train_demo; к validation применяется лишь transform.
X_demo_train = imputer.fit_transform(X_demo_train)
X_demo_val = imputer.transform(X_demo_val)

X_demo_train = scaler.fit_transform(X_demo_train)
X_demo_val = scaler.transform(X_demo_val)

# Формула логистической потери ниже ожидает метки классов −1 и +1.
y_demo_train_pm = np.where(y_demo_train.to_numpy() == 1, 1.0, -1.0)
y_demo_val_pm = np.where(y_demo_val.to_numpy() == 1, 1.0, -1.0)
```

Почему преобразования обучаются только на train_demo. Медианы и параметры масштабирования являются статистиками данных. Их оценивание по validation или test создало бы утечку информации.

Полная учебная реализация логистической регрессии и SGD

Листинг 12 — Функции потерь и обучения

```
def sigmoid(z):
    # Ограничение аргумента предотвращает переполнение функции exp.
    z = np.clip(np.asarray(z, dtype=float), -500.0, 500.0)
    return 1.0 / (1.0 + np.exp(-z))


def mean_logistic_loss(X, y_pm, w, b):
    # Отступ положителен, если объект классифицирован правильно.
    margins = y_pm * (X @ w + b)
    # logaddexp устойчиво вычисляет log(1 + exp(-margin)).
    return np.mean(np.logaddexp(0.0, -margins))


def train_logreg_sgd(
    X_train,
    y_train_pm,
    X_val,
    y_val_pm,
    *,
    eta=0.01,
    tau=1e-4,
    epochs=50,
    random_state=42,
):
    # Отдельный генератор случайных чисел делает перемешивание воспроизводимым.
    rng = np.random.default_rng(random_state)
    # Начинаем с нулевых весов и свободного коэффициента.
    w = np.zeros(X_train.shape[1], dtype=float)
    b = 0.0
    # История нужна для графиков сходимости и контроля переобучения.
    history = {"train_loss": [], "val_loss": [], "weight_norm": []}
```

```
    for _ in range(epochs):
        # В начале каждой эпохи случайно меняем порядок объектов.
        for i in rng.permutation(len(X_train)):
            x_i = X_train[i]
            y_i = y_train_pm[i]
            margin = y_i * (x_i @ w + b)

            # Градиент потери вычисляется по одному объекту.
            factor = -y_i * sigmoid(-margin)
            # L2-регуляризация добавляется только к градиенту весов.
            grad_w = factor * x_i + tau * w
            grad_b = factor

            # Одновременный шаг SGD для весов и свободного коэффициента.
            w -= eta * grad_w
            b -= eta * grad_b

        # После эпохи сохраняем train/validation loss и норму весов.
        history["train_loss"].append(
            mean_logistic_loss(X_train, y_train_pm, w, b)
        )
        history["val_loss"].append(
            mean_logistic_loss(X_val, y_val_pm, w, b)
        )
        history["weight_norm"].append(np.linalg.norm(w))

    return w, b, pd.DataFrame(history)
```

В строке factor = $-y_i\sigma(-M_i)$ вычисляется общий множитель градиента. Слагаемое tau · w реализует L2-регуляризацию; коэффициент b не регуляризуется.

Исследование темпа обучения и регуляризации

Листинг 13 — Сравнение нескольких значений eta

```
# Значения eta отличаются на порядок, чтобы увидеть разные режимы сходимости.
eta_values = [0.0001, 0.001, 0.01, 0.1]
eta_runs = {}

for eta in eta_values:
    # Все прочие параметры фиксированы: меняется только темп обучения.
    _, _, history = train_logreg_sgd(
        X_demo_train,
        y_demo_train_pm,
        X_demo_val,
        y_demo_val_pm,
        eta=eta,
        tau=1e-4,
        epochs=50,
        random_state=RANDOM_STATE,
    )
    eta_runs[eta] = history
    # На график выводим validation loss, а не тестовую метрику.
    plt.plot(history["val_loss"], label=f"eta={eta}")

plt.xlabel("Эпоха")
plt.ylabel("Validation log-loss")
plt.title("Влияние темпа обучения")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# Итоговая таблица дополняет график точными значениями последней эпохи.
eta_result = pd.DataFrame({
    eta: {
        "final_train_loss": h["train_loss"].iloc[-1],
```

```
        "final_val_loss": h["val_loss"].iloc[-1],
        "weight_norm": h["weight_norm"].iloc[-1],
    }
    for eta, h in eta_runs.items()
}).T
display(eta_result.round(4))
```

Листинг 14 — Сравнение нескольких значений tau

```
# Лучшее eta выбираем только по validation loss предыдущего эксперимента.
best_eta = eta_result["final_val_loss"].idxmin()
tau_values = [0.0, 1e-4, 1e-2]
tau_result = []

for tau in tau_values:
    # При фиксированном eta сравниваем только силу L2-регуляризации.
    w_sgd, b_sgd, history = train_logreg_sgd(
        X_demo_train,
        y_demo_train_pm,
        X_demo_val,
        y_demo_val_pm,
        eta=float(best_eta),
        tau=tau,
        epochs=50,
        random_state=RANDOM_STATE,
    )
    # Сохраняем качество и норму весов: обе величины важны для вывода.
    tau_result.append({
        "tau": tau,
        "val_loss": history["val_loss"].iloc[-1],
        "weight_norm": np.linalg.norm(w_sgd),
    })

# Каждая строка итоговой таблицы соответствует одному значению tau.
tau_result = pd.DataFrame(tau_result)
display(tau_result.round(4))
```

В отчёте необходимо связать форму кривых с теорией: малый шаг обычно даёт медленное снижение loss, а чрезмерный — колебания или рост loss. При увеличении tau норма весов обычно уменьшается; влияние на validation loss определяется данными.

Для своего варианта. Если задача многоклассовая, выберите один класс как +1, остальные объедините в −1. Если числовых признаков нет, выполните one-hot кодирование на train_demo и используйте не более 20 полученных столбцов.

Полный preprocessing pipeline

Листинг 15 — ColumnTransformer и SGDClassifier

```
# Сначала разделяем столбцы по способу обработки.
numeric_features = X_train.select_dtypes(include="number").columns.tolist()
categorical_features = [
    column for column in X_train.columns
    if column not in numeric_features
]

# Числовые признаки: заполняем пропуски медианой и масштабируем.
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

# Категориальные признаки: заполняем модой и выполняем one-hot кодирование.
# handle_unknown="ignore" позволяет обрабатывать новые категории на test.
categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
```

```
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])

# ColumnTransformer применяет две ветви обработки параллельно.
preprocessor = ColumnTransformer([
    ("num", numeric_pipeline, numeric_features),
    ("cat", categorical_pipeline, categorical_features),
])

# Общий Pipeline гарантирует, что preprocessing обучается заново внутри каждого CV-фолда.
sgd_pipeline = Pipeline([
    ("prep", preprocessor),
    ("model", SGDClassifier(
        # log_loss соответствует логистической регрессии.
        loss="log_loss",
        penalty="l2",
        learning_rate="constant",
        max_iter=2000,
        tol=1e-4,
        random_state=RANDOM_STATE,
    )),
])
```

Проверка типов. Автоматическое определение по dtype не распознаёт категориальный смысл числовых кодов. Если столбец содержит коды категорий, его следует явно добавить в categorical_features и удалить из numeric_features.

Подбор гиперпараметров и сравнение моделей

Листинг 16 — GridSearchCV для SGDClassifier

```
# Префикс model__ обращается к параметрам шага "model" внутри Pipeline.
sgd_param_grid = {
    "model__eta0": [0.001, 0.01, 0.1],
    "model__alpha": [1e-6, 1e-4, 1e-2],
}

# GridSearchCV проверяет каждую пару eta0/alpha на одной и той же CV-схеме.
sgd_search = GridSearchCV(
    estimator=sgd_pipeline,
    param_grid=sgd_param_grid,
    scoring=SCORING,
    cv=cv,
    n_jobs=-1,
    return_train_score=True,
)
# В fit передаём исходные X_train: preprocessing выполняется внутри Pipeline.
sgd_search.fit(X_train, y_train)

print("Лучшие параметры SGD:", sgd_search.best_params_)
print("Лучший CV macro-F1:", round(sgd_search.best_score_, 4))

# cv_results_ содержит метрики всех комбинаций, а не только победителя.
sgd_cv_table = pd.DataFrame(sgd_search.cv_results_)[[
    "param_model__eta0",
    "param_model__alpha",
    "mean_train_score",
    "mean_test_score",
    "std_test_score",
    "rank_test_score",
]].sort_values("rank_test_score")
display(sgd_cv_table.round(4))
```

Листинг 17 — Тепловая карта результатов CV без seaborn

```
# Преобразуем длинную таблицу результатов в матрицу alpha × eta0.
pivot = sgd_cv_table.pivot_table(
    index="param_model__alpha",
    columns="param_model__eta0",
    values="mean_test_score",
)

# Оттенок серого кодирует средний CV macro-F1.
fig, ax = plt.subplots(figsize=(6, 4))
image = ax.imshow(pivot.to_numpy(), cmap="Greys", aspect="auto")
ax.set_xticks(range(len(pivot.columns)), labels=pivot.columns)
ax.set_yticks(range(len(pivot.index)), labels=pivot.index)
ax.set_xlabel("eta0")
ax.set_ylabel("alpha")
ax.set_title("CV macro-F1 для SGDClassifier")
# Шкала справа показывает соответствие оттенка численному значению метрики.
fig.colorbar(image, ax=ax, label="macro-F1")
plt.tight_layout()
plt.show()
```

Листинг 18 — Подбор C для LogisticRegression

```
# clone создаёт независимую копию уже описанного preprocessing.
logreg_pipeline = Pipeline([
    ("prep", clone(preprocessor)),
    ("model", LogisticRegression(
        max_iter=3000,
        random_state=RANDOM_STATE,
    )),
])

# Для LogisticRegression подбираем обратную силу регуляризации C.
logreg_search = GridSearchCV(
    estimator=logreg_pipeline,
    param_grid={"model__C": [0.01, 0.1, 1.0, 10.0]},
    scoring=SCORING,
    cv=cv,
    n_jobs=-1,
)
logreg_search.fit(X_train, y_train)

# Сводим baseline и две обучаемые модели в одну таблицу результатов CV.
cv_comparison = pd.DataFrame([
    {
        "model": "DummyClassifier",
        "best_params": "strategy=most_frequent",
        "cv_macro_f1": baseline_summary["f1_macro_cv_mean"],
    },
    {
        "model": "SGDClassifier",
        "best_params": str(sgd_search.best_params_),
        "cv_macro_f1": sgd_search.best_score_,
    },
    {
        "model": "LogisticRegression",
        "best_params": str(logreg_search.best_params_),
        "cv_macro_f1": logreg_search.best_score_,
    },
]).sort_values("cv_macro_f1", ascending=False)

display(cv_comparison.round(4))

# DummyClassifier остаётся ориентиром, но не кандидатом на основную модель.
candidate_comparison = cv_comparison[
    cv_comparison["model"] != "DummyClassifier"
].reset_index(drop=True)
```

Модель для финальной оценки выбирается только по cv_macro_f1 и содержательным соображениям, зафиксированным до обращения к test. Значения eta0, alpha и C нельзя изменять после просмотра тестовых метрик.

Финальная оценка на test

Листинг 19 — Функция вычисления метрик

```
def evaluate_classifier(name, estimator, X_eval, y_eval):
    # Классовые метки и вероятности вычисляются один раз для всех метрик.
    prediction = estimator.predict(X_eval)
    probabilities = estimator.predict_proba(X_eval)
    classes = estimator.classes_

    if len(classes) == 2:
        # Для бинарного ROC-AUC явно кодируем второй класс как положительный.
        positive_class = classes[1]
        y_binary = (np.asarray(y_eval) == positive_class).astype(int)
        auc = roc_auc_score(y_binary, probabilities[:, 1])
    else:
        # Для многоклассовой задачи используем One-vs-Rest и macro-усреднение.
        auc = roc_auc_score(
            y_eval,
            probabilities,
            labels=classes,
            multi_class="ovr",
            average="macro",
        )

    # Единый словарь позволяет затем собрать результаты моделей в DataFrame.
    return {
        "model": name,
        "accuracy": accuracy_score(y_eval, prediction),
        "precision_macro": precision_score(
            y_eval, prediction, average="macro", zero_division=0
        ),
        "recall_macro": recall_score(
            y_eval, prediction, average="macro", zero_division=0
        ),
        "f1_macro": f1_score(
            y_eval, prediction, average="macro", zero_division=0
        ),
        "roc_auc": auc,
    }
```

Листинг 20 — Однократное тестирование зафиксированных моделей

```
# Baseline дообучается на всей обучающей части перед единственной оценкой test.
baseline.fit(X_train, y_train)

# best_estimator_ уже содержит параметры, выбранные только по CV.
final_models = {
    "DummyClassifier": baseline,
    "SGDClassifier": sgd_search.best_estimator_,
    "LogisticRegression": logreg_search.best_estimator_,
}

# Тестовые метрики всех зафиксированных моделей вычисляются одним проходом.
test_results = pd.DataFrame([
    evaluate_classifier(name, model, X_test, y_test)
    for name, model in final_models.items()
])
display(test_results.round(4))

# Основную модель выбирали по CV; здесь только извлекаем зафиксированный выбор.
```

```
selected_name = candidate_comparison.iloc[0]["model"]
selected_model = final_models[selected_name]
selected_prediction = selected_model.predict(X_test)

print(classification_report(y_test, selected_prediction, zero_division=0))

# Нормировка по строкам показывает доли прогнозов внутри каждого истинного класса.
ConfusionMatrixDisplay.from_estimator(
    selected_model,
    X_test,
    y_test,
    normalize="true",
    cmap="Greys",
)
plt.title(f"Нормированная confusion matrix: {selected_name}")
plt.show()
```

После открытия test. Тестовые результаты используются только для финального отчёта. Даже если одна из моделей показала неожиданно низкое качество, возвращаться к подбору eta0, alpha, C или признаков по этим значениям нельзя.

Анализ ошибок и коэффициентов

Листинг 21 — Таблица наиболее уверенных ошибок

```
# Начинаем с исходных признаков тестовых объектов.
error_table = X_test.copy()
# Добавляем истинный класс и прогноз выбранной модели.
error_table["true"] = np.asarray(y_test)
error_table["predicted"] = selected_prediction

# Уверенность определяем как максимальную прогнозируемую вероятность класса.
selected_probabilities = selected_model.predict_proba(X_test)
error_table["confidence"] = selected_probabilities.max(axis=1)

# Оставляем только ошибки и сортируем их от наиболее уверенных к менее уверенным.
errors = error_table[
    error_table["true"] != error_table["predicted"]
].sort_values("confidence", ascending=False)

display(errors.head(10))
```

Листинг 22 — Извлечение наиболее значимых коэффициентов

```
# Получаем имена признаков после импутации и one-hot кодирования.
feature_names = selected_model.named_steps["prep"].get_feature_names_out()
# Обращаемся к обученной линейной модели внутри Pipeline.
model = selected_model.named_steps["model"]

# Для бинарной задачи coef_ содержит одну строку.
# Для многоклассовой задачи укажите анализируемый класс.
class_index = 0
coefficients = model.coef_[class_index]

# Абсолютное значение используется только для ранжирования силы коэффициентов.
coefficient_table = pd.DataFrame({
    "feature": feature_names,
    "coefficient": coefficients,
    "abs_coefficient": np.abs(coefficients),
}).sort_values("abs_coefficient", ascending=False)

# Знак коэффициента нужно интерпретировать отдельно от его абсолютной величины.
display(coefficient_table.head(15))
```

Необходимо описать 5–10 характерных ошибок, определить наиболее часто смешиваемые классы и предложить не менее трёх содержательных причин: пересечение классов, пропуски, шум, дисбаланс, недостаточность признаков или линейная неразделимость.

Образец формулировки выводов

Выводы должны содержать наблюдение, численное подтверждение и интерпретацию. Корректный шаблон формулировки:

- «При увеличении eta с <значение 1> до <значение 2> validation log-loss <изменение>; это указывает на <интерпретация>».
- «При tau = <значение> норма весов уменьшилась с <…> до <…>, а validation loss <изменение>; следовательно, регуляризация <вывод>».
- «Лучшая модель по CV — <название>, macro-F1 = <…>; на test получено <…>, разница <…>».
- «Чаще всего смешивались классы <…> и <…>; анализ объектов показывает <причина>».
- «Линейная модель ограничена <…>; следующим разумным шагом является <метод курса>».

Недостаточная формулировка. Фраза «в работе была изучена логистическая регрессия и получены навыки машинного обучения» не содержит результата эксперимента и не засчитывается как вывод.

Что изменить для своего варианта

1. заменить DATA_ID и RANDOM_STATE;
2. загружать все содержательные признаки варианта, а не список features из Titanic;
3. проверить фактические типы признаков и вручную исправить признаки-коды категорий;
4. для собственной реализации выбрать бинарную подзадачу и не более 20 признаков;
5. сохранить неизменными правила разделения Train/CV/Test;
6. выполнить подбор eta0, alpha и C только по обучающей части;
7. получить собственные таблицы, графики, confusion matrix и выводы.

8. Задание по индивидуальному варианту

Этап 1. Загрузка и первичный анализ

1. загрузить набор варианта и вывести название, data_id и размерность;
2. вывести первые строки, типы столбцов и число уникальных значений;
3. определить числовые, категориальные, порядковые и служебные признаки;
4. посчитать пропуски и долю объектов хотя бы с одним пропуском;
5. исследовать абсолютные частоты и доли классов;

6. построить распределение классов и не менее одной диаграммы по признакам;
7. заполнить карточку данных и назвать 2–3 потенциальные сложности.

Этап 2. Экспериментальный протокол

1. задать RANDOM_STATE = 1000 + номер варианта;
2. отделить 20% данных в test с stratify=y;
3. создать StratifiedKFold: 5 фолдов, shuffle=True;
4. принять macro-F1 основной метрикой подбора;
5. зафиксировать назначение Train, CV и Test в отдельной текстовой ячейке notebook.

Этап 3. Baseline

1. оценить DummyClassifier(strategy="most_frequent") посредством той же CV;
2. вычислить средние Accuracy и macro-F1;
3. объяснить, почему baseline необходим для интерпретации основной модели.

Этап 4. Собственная реализация SGD

1. сформировать бинарную подзадачу; для многоклассового варианта использовать схему «один класс против остальных»;
2. выбрать не более 20 признаков и сформировать train_demo / val_demo только из X_train;
3. обучить импутацию и масштабирование только на train_demo;
4. адаптировать функции из листинга 12 и проверить формулу одного SGD-шага;
5. обучать 30–50 эпох со случайным перемешиванием объектов;
6. сравнить минимум три значения eta и не менее двух значений tau;
7. построить train/validation loss и сравнить нормы весов.

Этап 5. Библиотечные модели и подбор

1. собрать ColumnTransformer для числовых и категориальных признаков;
2. включить preprocessing и SGDClassifier(loss="log_loss") в Pipeline;
3. подобрать eta0 $\in \{0.001, 0.01, 0.1\}$ и alpha $\in \{10^{-6}, 10^{-4}, 10^{-2}\}$;
4. сохранить mean_test_score, std_test_score и построить график результатов CV;
5. построить второй Pipeline с LogisticRegression и подобрать C $\in \{0.01, 0.1, 1, 10\}$;
6. выбрать модель по CV до обращения к тестовой выборке.

Этап 6. Финальная оценка и анализ ошибок

1. однократно вычислить на test Accuracy, macro-Precision, macro-Recall, macro-F1 и ROC-AUC;
2. построить нормированную confusion matrix;

3. представить результаты baseline, SGDClassifier и LogisticRegression в одной таблице;
4. исследовать 5–10 ошибочно классифицированных объектов;
5. сформулировать не менее трёх причин ошибок и 5–8 итоговых выводов.

Сокращённая сетка. Если полный подбор не укладывается в отведённое время, сначала используйте eta0 $\in$ {0.001, 0.01} и alpha $\in$ {10^{-4}, 10^{-2}}. Последовательность «выбор по CV → одна финальная оценка на test» изменять нельзя.

9. Контрольные вопросы

1. Что такое обучение с учителем и чем классификация отличается от регрессии?
2. Что называется эмпирическим риском? Какую роль играет функция потерь?
3. Чем функция потерь отличается от метрики качества?
4. Что называется отступом объекта в линейной классификации?
5. Почему логистическая функция потерь удобна для градиентной оптимизации?
6. Чем полный градиентный спуск отличается от SGD?
7. Что происходит при слишком малом и слишком большом learning rate?
8. Зачем масштабировать числовые признаки перед линейной моделью, обучаемой SGD?
9. Как L2-регуляризация влияет на веса модели?
10. Чем L1-регуляризация концептуально отличается от L2?
11. Почему гиперпараметры нельзя выбирать по тестовой выборке?
12. Что такое data leakage? Приведите два примера для этой работы.
13. Почему preprocessing следует включать в Pipeline при кросс-валидации?
14. Когда Accuracy может давать вводящую в заблуждение оценку?
15. Как интерпретируются Precision и Recall?
16. Что показывает ROC-AUC и почему метрика не зависит от одного фиксированного порога?
17. Почему macro-F1 удобна для сравнения многоклассовых вариантов?
18. Чем alpha в SGDClassifier отличается от C в LogisticRegression?
19. Почему SGDClassifier и LogisticRegression могут решать близкую задачу, но давать разные результаты?
20. Каковы главные ограничения линейного классификатора на вашем наборе?
21. Какое eta оказалось лучшим в собственной реализации и почему?
22. Как регуляризация изменила норму весов и качество на validation/test?

При защите необходимо быть готовым ответить на 2–3 вопроса, включая вопрос, связанный с фактическими результатами собственного варианта.

10. Требования к отчёту

Состав сдаваемых материалов

- Jupyter Notebook (.ipynb), воспроизводимый командой Restart Kernel → Run All;
- отчёт в формате PDF или DOCX;
- исходный набор данных — только если он был изменён или дополнен студентом.

Рекомендуемая структура отчёта

1. титульный лист;
2. тема и цель работы;
3. номер варианта и карточка набора данных;
4. краткие теоретические сведения;
5. экспериментальный протокол;
6. baseline;
7. собственная реализация SGD и графики loss;
8. preprocessing pipeline;
9. подбор eta0, alpha и C;
10. сравнение моделей;
11. финальная оценка на test и confusion matrix;
12. анализ ошибок;
13. выводы.

Требования к оформлению

- таблицы и рисунки имеют номер и содержательную подпись;
- оси графиков подписаны, единицы и смысл величин указаны;
- каждая таблица и каждый рисунок упомянуты и интерпретированы в тексте;
- в отчёт включаются только ключевые фрагменты кода, полный код остаётся в notebook;
- рекомендуемый объём основного текста — 5–8 страниц без листингов и титульного листа.

Требования к коду

- нет скрытых ручных шагов и зависимости от порядка случайного запуска ячеек;
- зафиксирован random_state;
- нет абсолютных путей;
- preprocessing не обучается на test;
- для библиотечной модели используется Pipeline / ColumnTransformer;

- повторяющиеся действия вынесены в функции, если это улучшает понятность;
- графики строятся из рассчитанных данных.

11. Критерии оценивания

Таблица 3 — Критерии оценивания

Критерий	Баллы
Корректная постановка задачи и работа с данными	5
Предварительная обработка и отсутствие data leakage	10
Построение и оценивание baseline	5
Корректность собственной реализации SGD	15
Корректность основной библиотечной модели	10
Исследование eta0 / alpha и организация model selection	15
Метрики, таблицы, визуализации и анализ ошибок	10
Интерпретация результатов и качество выводов	10
Воспроизводимость, качество кода и отчёта	5
Ответы при защите	15
Итого	100

- 90–100 баллов — отлично;
- 75–89 баллов — хорошо;
- 60–74 балла — удовлетворительно;
- менее 60 баллов — работа подлежит доработке.

Высокая Accuracy сама по себе не компенсирует некорректный эксперимент.

12. Существенные ошибки и основания для доработки

Работа требует исправления независимо от итоговой метрики, если обнаружено хотя бы одно из следующих нарушений:

- модель или preprocessing обучались с использованием X_test;
- eta0, alpha, C, признаки или иные гиперпараметры выбирались по test;
- нормировка или импутация выполнены на всей выборке до train/test split;
- при CV preprocessing выполняется вне Pipeline и использует статистики всех фолдов;
- применена некорректная метрика или дана неверная интерпретация;
- отсутствует baseline;
- собственная реализация SGD не соответствует заданному обновлению;

- notebook не выполняется после Restart Kernel → Run All;
- таблицы и графики приведены без объяснения;
- выводы не связаны с численными результатами.

13. Дополнительное задание

По желанию выполните одно задание повышенной сложности:

- сравнить L1- и L2-регуляризацию; для L1 оценить долю нулевых коэффициентов;
- для бинарной задачи исследовать Precision/Recall при изменении порога;
- построить learning curve по объёму обучающей выборки;
- сравнить constant и adaptive/optimal learning rate schedule;
- для многоклассовой задачи сравнить macro- и weighted-метрики.

Приложение А. Таблица вариантов

В таблице приведены 30 классификационных наборов OpenML. Идентификатор data_id используется для однозначной загрузки данных. Если номер варианта повторяется, индивидуальность эксперимента сохраняется за счёт RANDOM_STATE и самостоятельного анализа результатов.

Таблица 4 — Варианты исходных данных

Вариант	data_id	Набор данных	Локальная копия
1	11	balance-scale	variant_01.csv
2	14	mfeat-fourier	variant_02.csv
3	15	breast-w	variant_03.csv
4	16	mfeat-karhunen	variant_04.csv
5	18	mfeat-morphological	variant_05.csv
6	22	mfeat-zernike	variant_06.csv
7	23	cmc	variant_07.csv
8	29	credit-approval	variant_08.csv
9	31	credit-g	variant_09.csv
10	37	diabetes	variant_10.csv
11	50	tic-tac-toe	variant_11.csv
12	54	vehicle	variant_12.csv
13	188	eucalyptus	variant_13.csv
14	458	analcatdata_authorship	variant_14.csv
15	469	analcatdata_dmft	variant_15.csv
16	1049	pc4	variant_16.csv
17	1050	pc3	variant_17.csv
18	1063	kc2	variant_18.csv
19	1068	pc1	variant_19.csv
20	1462	banknote-authentication	variant_20.csv
21	1464	blood-transfusion-service-center	variant_21.csv
22	1480	ilpd	variant_22.csv
23	1494	qsar-biodeg	variant_23.csv
24	1510	wdbc	variant_24.csv
25	6332	cylinder-bands	variant_25.csv
26	23381	dresses-sales	variant_26.csv
27	40966	MiceProtein	variant_27.csv
28	40975	car	variant_28.csv
29	40982	steel-plates-fault	variant_29.csv
30	40994	climate-model-simulation-crashes	variant_30.csv

Приложение Б. Минимальный шаблон отчёта

Цель работы

Сформулировать цель своими словами в 2–4 предложениях.

Вариант и данные

Указать вариант, data_id, название и заполнить карточку данных.

Экспериментальный протокол

Указать RANDOM_STATE, test_size, CV, метрику и правила preprocessing.

Baseline

Привести CV-метрики baseline и интерпретировать уровень качества.

SGD с нуля

Привести формулы, ключевой код, train/validation loss и влияние tau.

Библиотечные модели

Описать Pipeline, сетки параметров и результаты CV.

Финальный test

Привести таблицу метрик и confusion matrix.

Анализ ошибок

Показать характерные ошибки и объяснить возможные причины.

Выводы

Сформулировать 5–8 выводов, содержащих численные результаты.